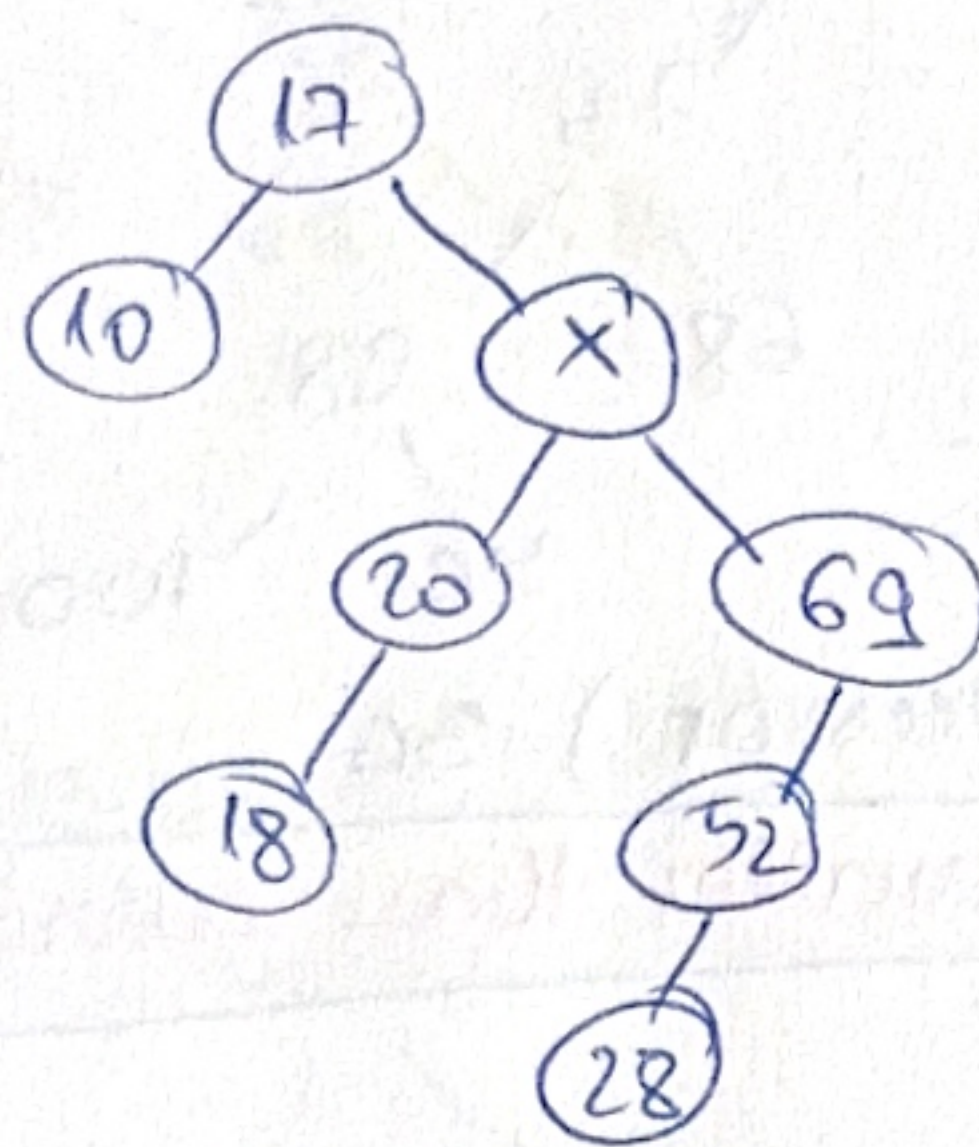


Kock Exam ①

①. ①.1. what node is x? (TREE) e 26



c) d) → not valid because ...

①.2. $12m^3 + m \log_2 m + 52$.

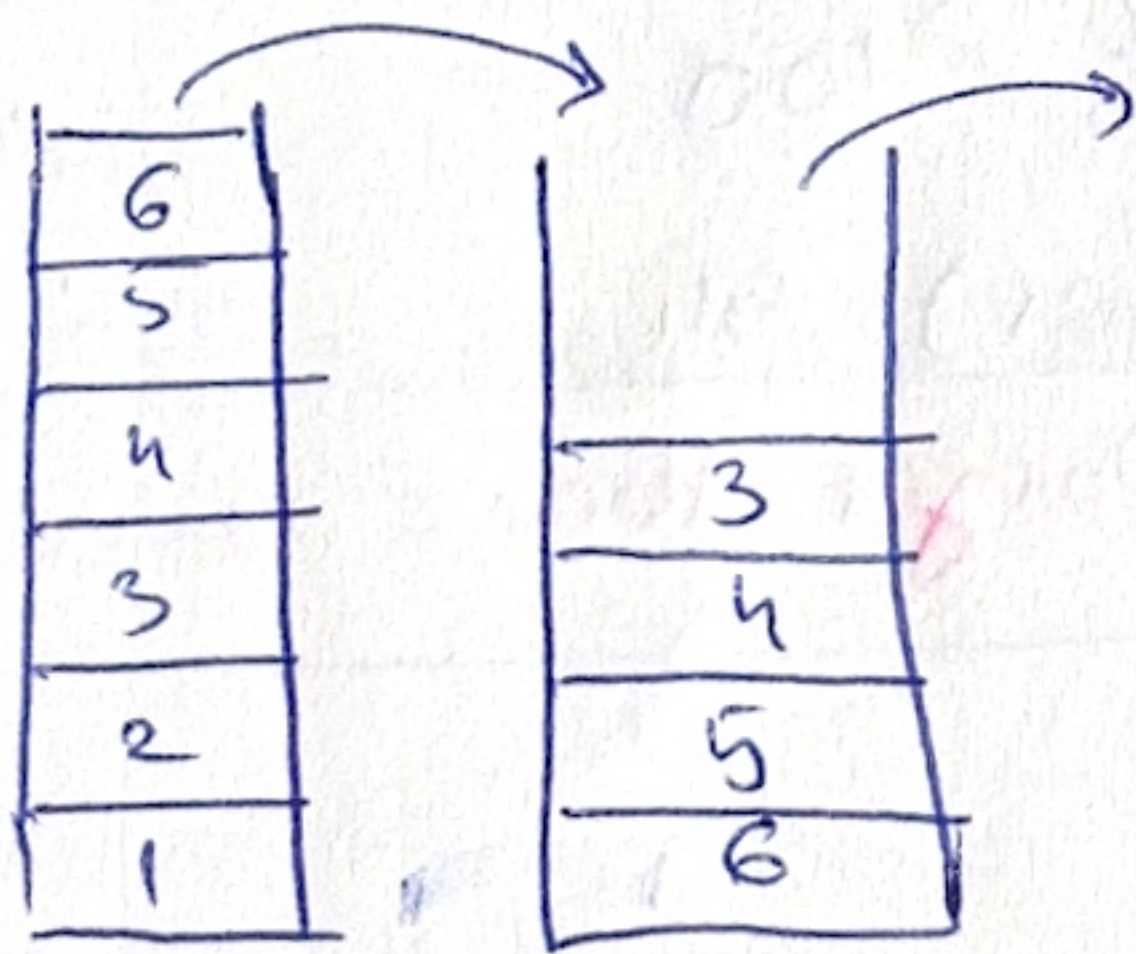
b) $\Omega(1)$

$$\Omega \leq \Theta \leq O$$

c) $O(m^4)$

a) $\Omega(m^3)$

①.3.



3

c) 3

①.4. 4 6 7 + 1 - 2 5 * + +

Token

4

6

7

+

1

-

2

5

*

+

+

Stack

4

4, 6

4, 6, 7

4, 13

4, 12, 1

4, 12

4, 12, 2

4, 12, 2, 5

4, 12, 10

4, 22

26

$$\Rightarrow 6 + 7 = 13$$

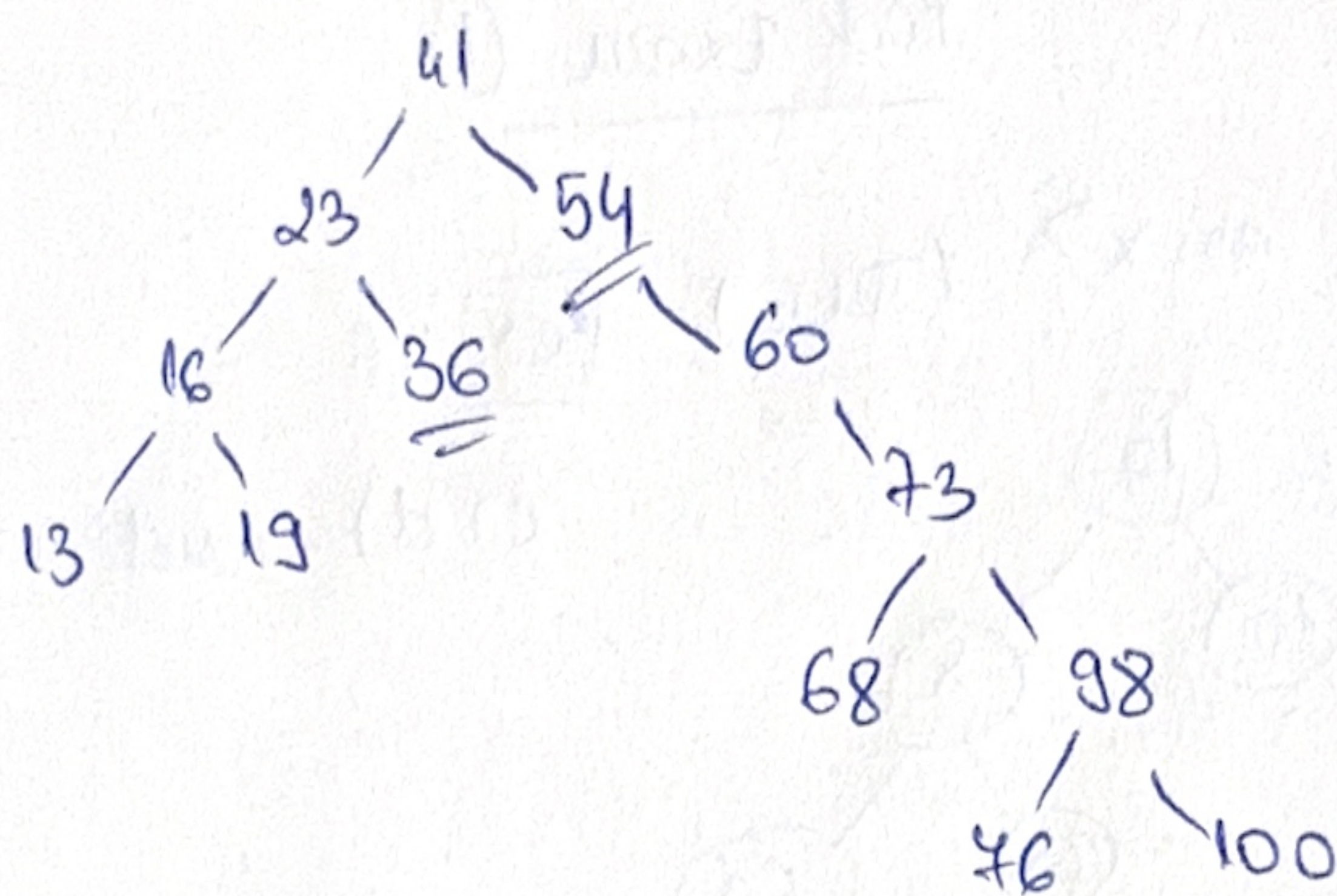
$$\Rightarrow 13 - 1 = 12$$

$$\Rightarrow 2 * 5 = 10$$

$$\Rightarrow 12 + 10 = 22$$

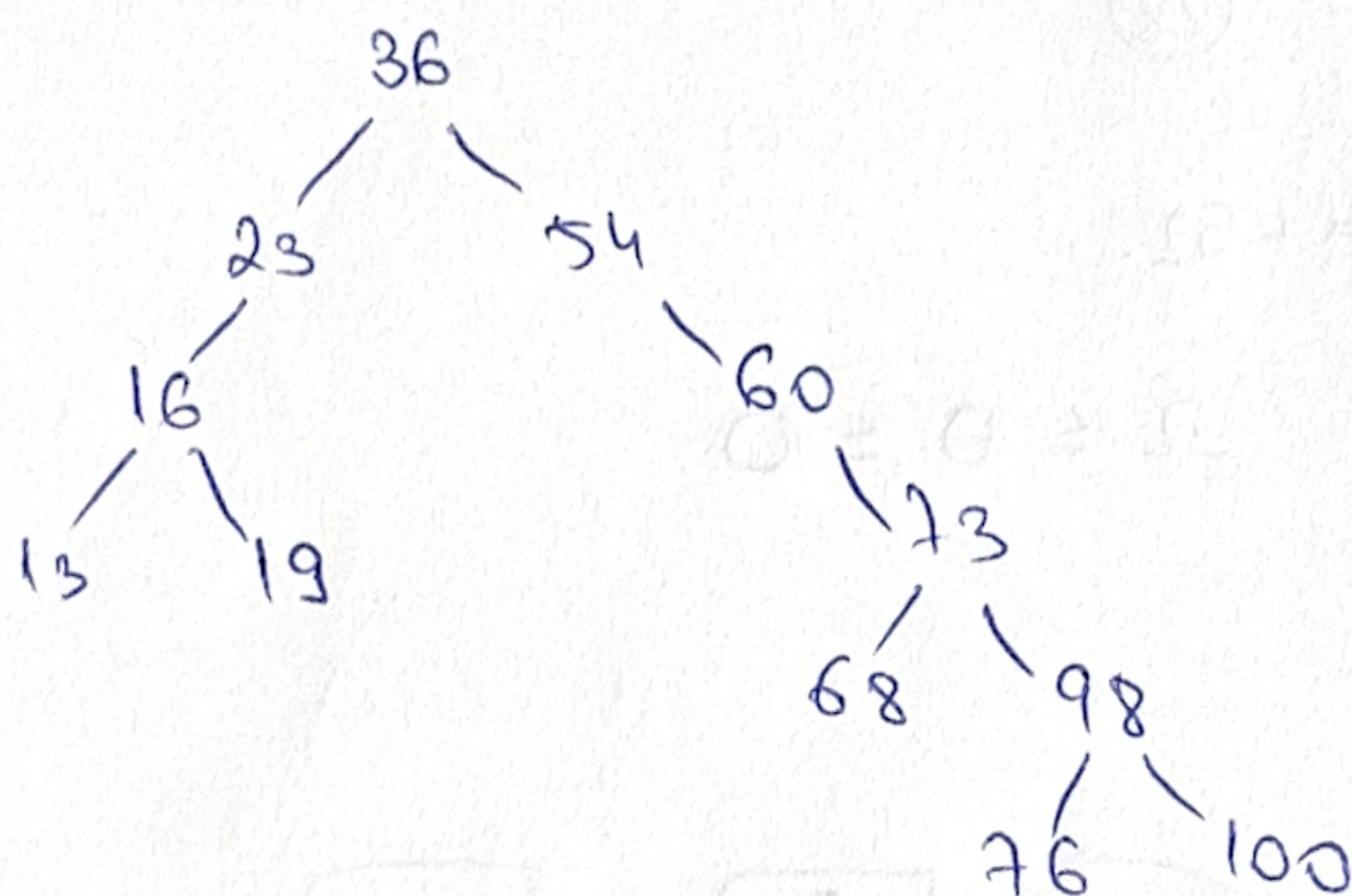
$$\Rightarrow 4 + 22 = 26$$

②.



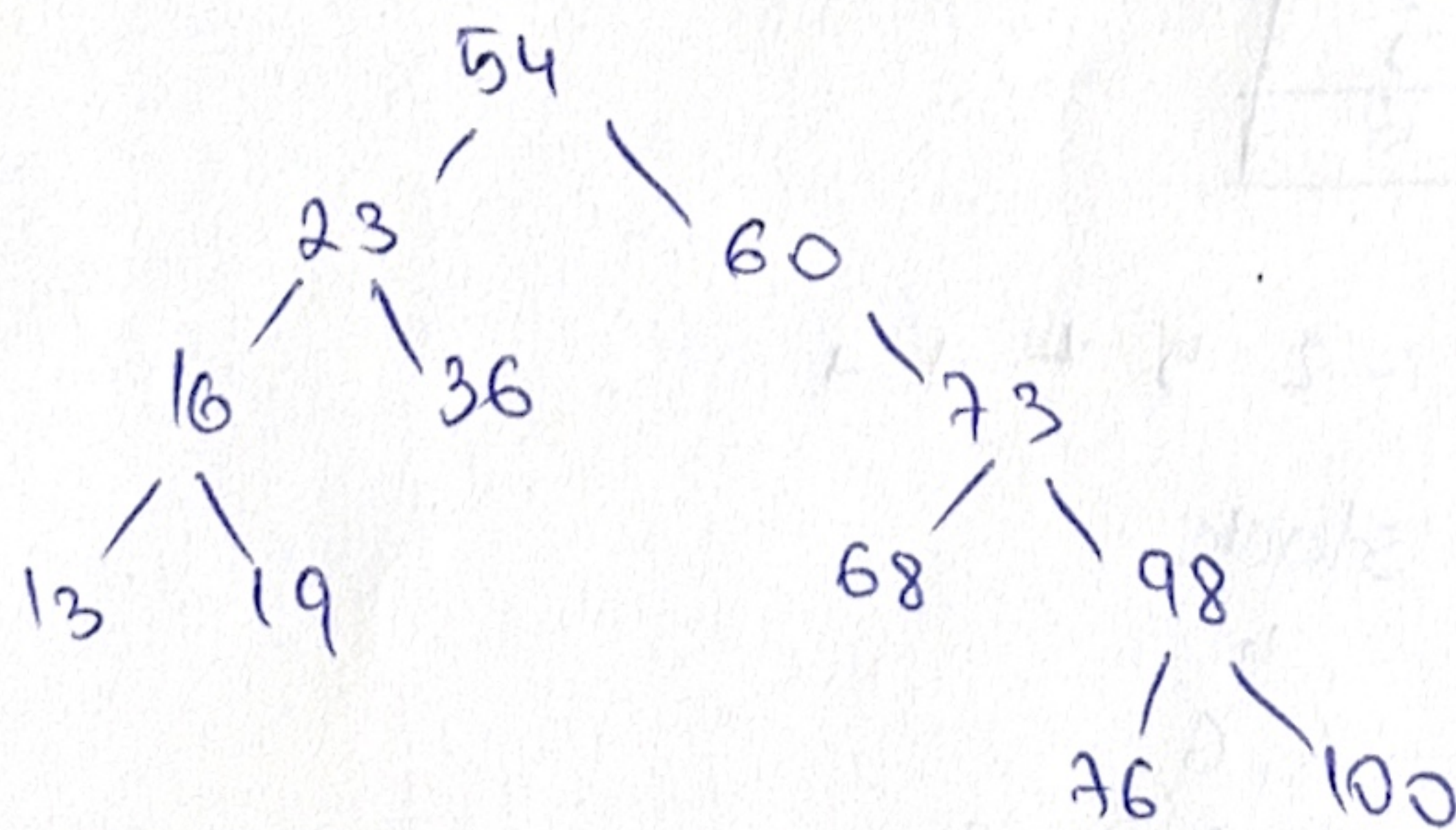
• Removal of 41 (replace with predecessor) : 36

predecessor = largest element smaller than current



• Removal of 41 (repl. with successor) : 54

successor = smallest element larger than current



②.2 HT, $m=8$, separate chaining

(*) position = root of a BST.

ADD: 8, 99, 40, 19, 56, 16, 17, 28, 62.

Load factor of the HT?

• $8 \% 8 = 0 = h(8)$

• $99 \% 8 = 3 = h(99)$

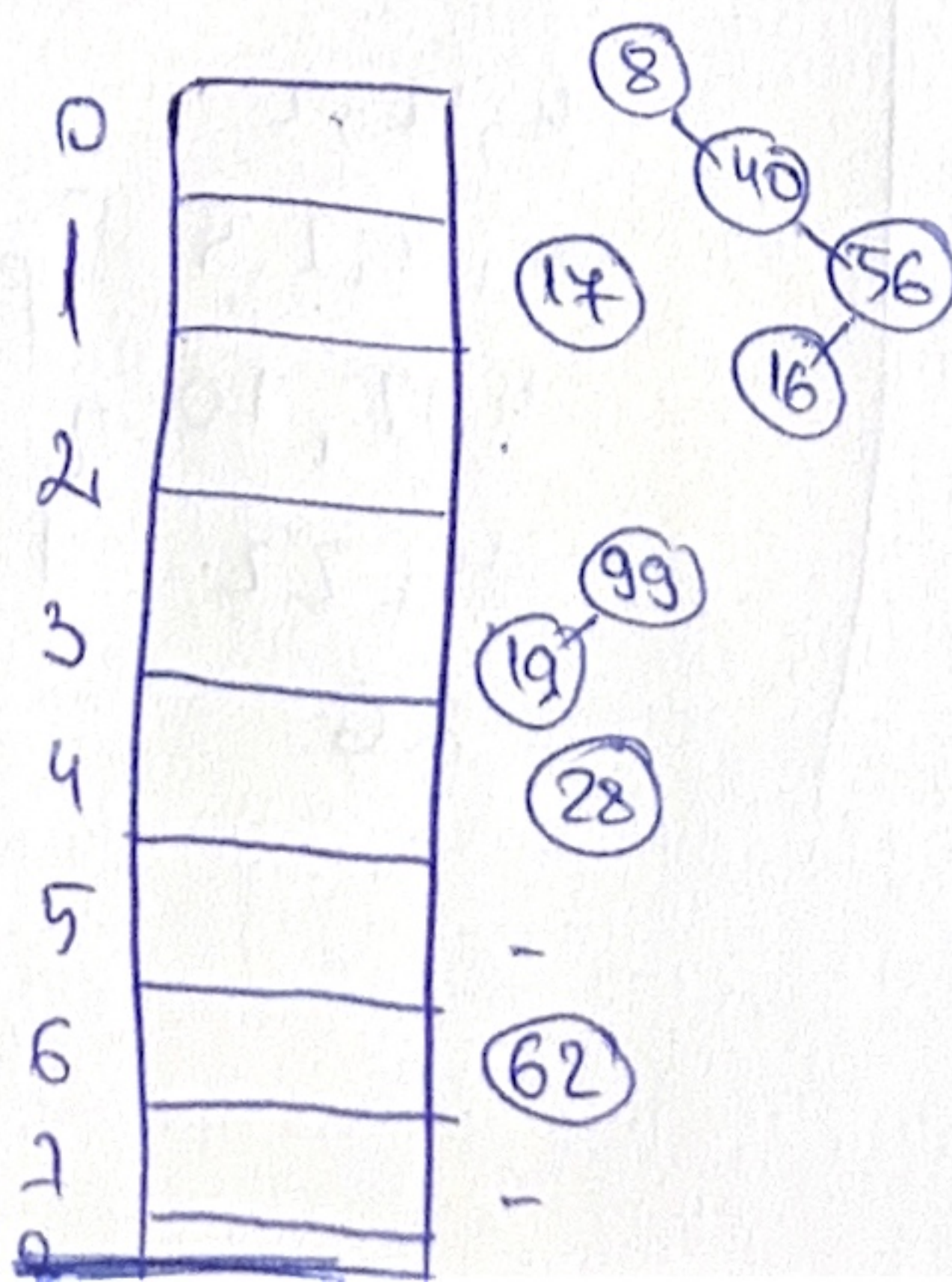
• $40 \% 8 = 0 = h(40)$

• $19 \% 8 = 3 = h(19)$

• $56 \% 8 = 0$ $28 \% 8 = 4$

• $16 \% 8 = 0$ $62 \% 8 = 6$

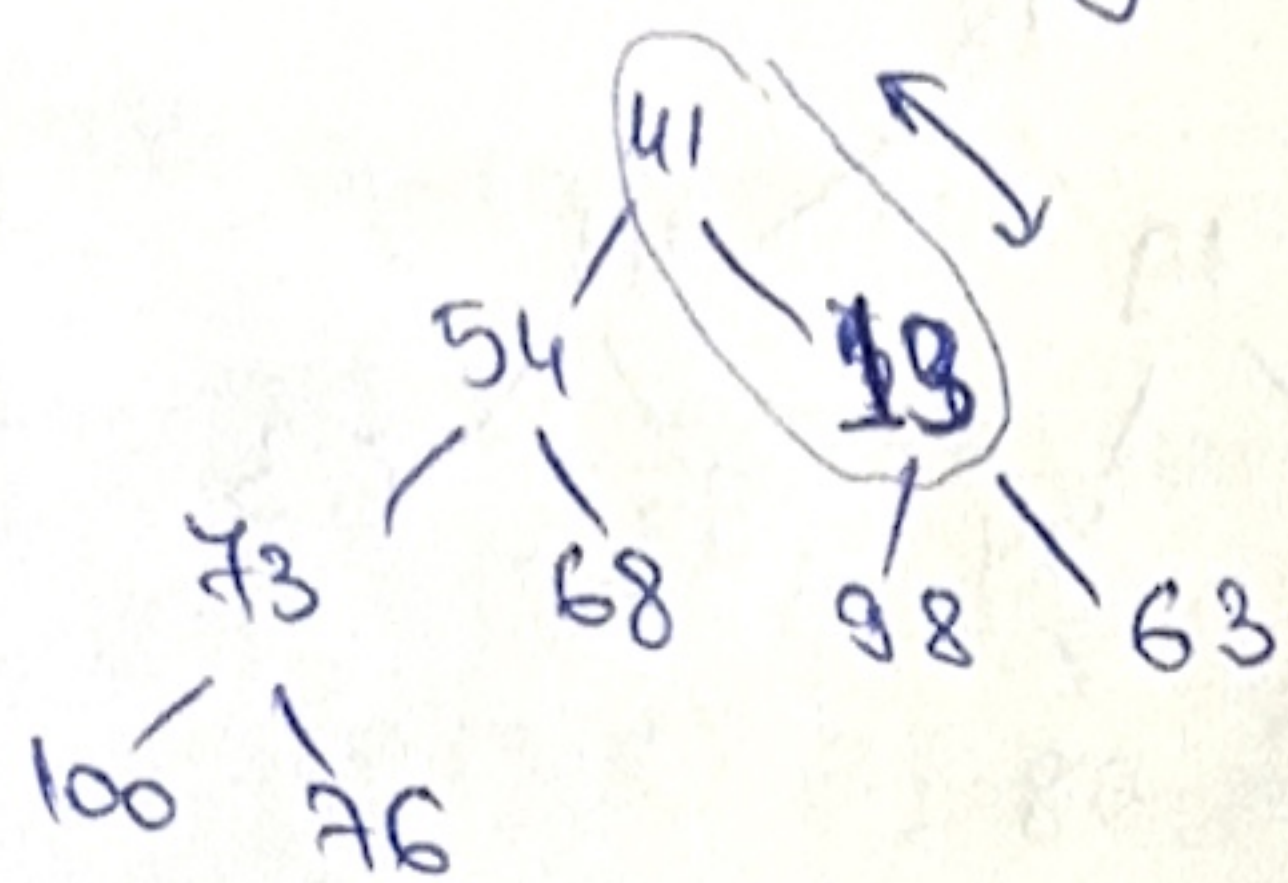
• $17 \% 8 = 1$



$\frac{12}{8} = 1.5$

$\alpha = \frac{n}{m} = \frac{9}{8} \rightarrow \text{load factor}$

Is this array a Binary Heap? Transform.

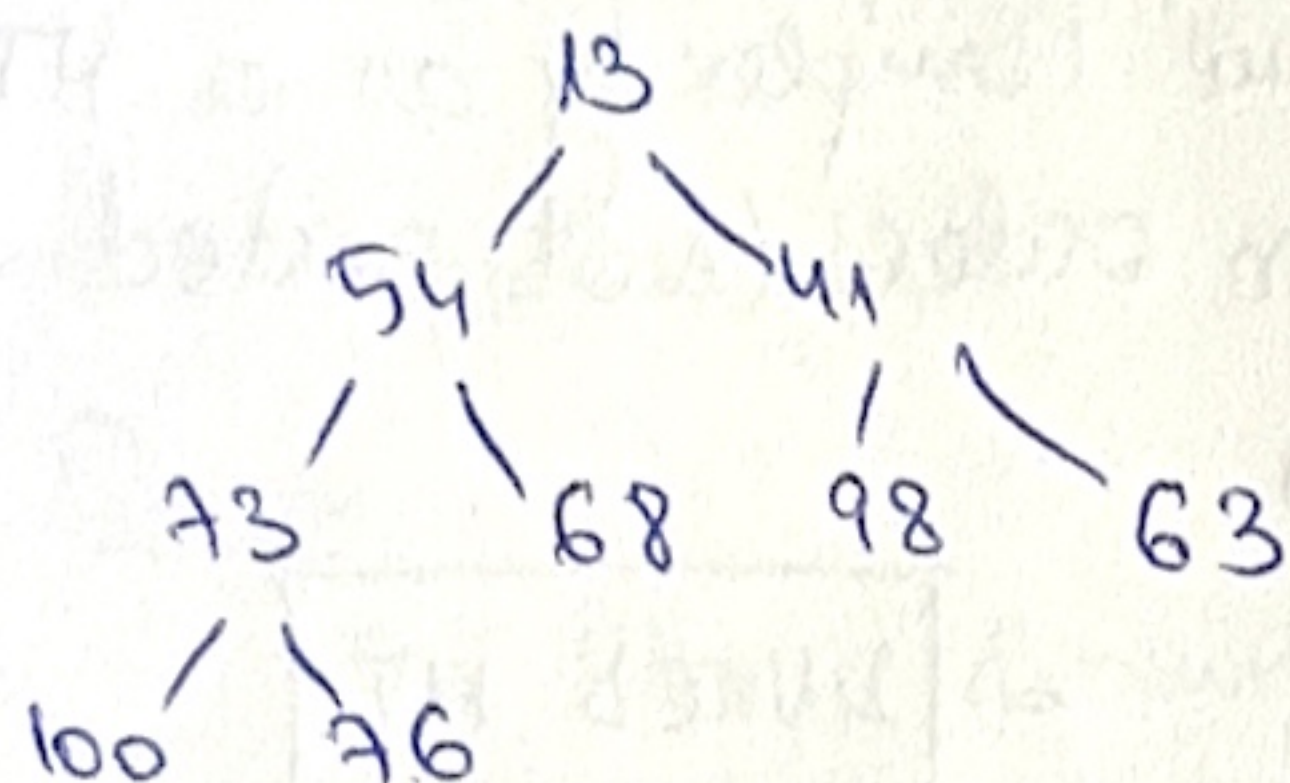


→ not a heap because:

1. it does not respect the min-heap property ($41 > 13$)

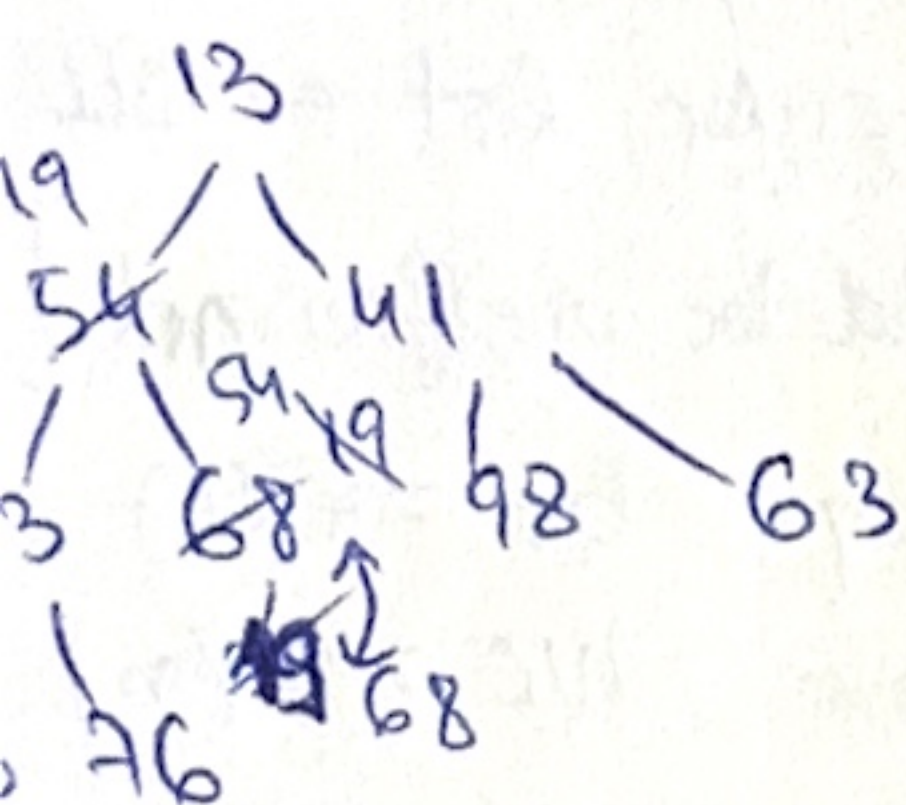
→ it does respect the structure of a heap.

⇓

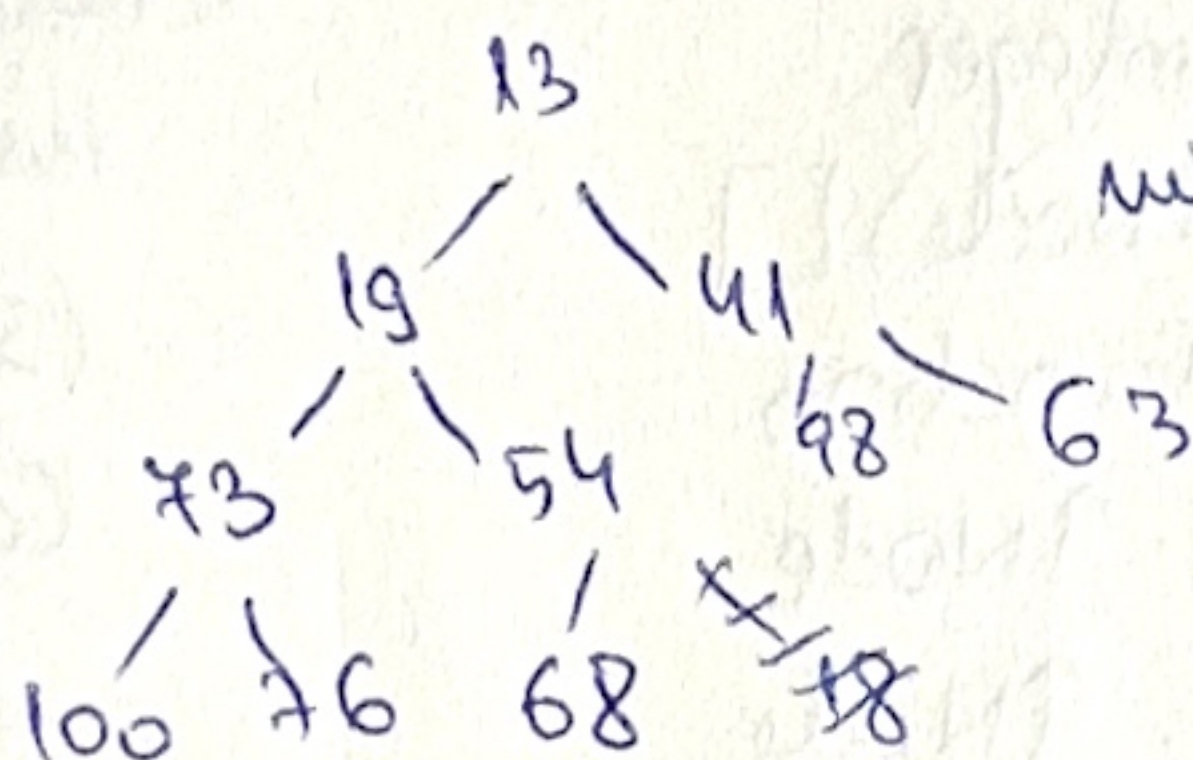


⇒ [13, 54, 41, 73, 68, 98, 63, 100, 76].

• add 19:

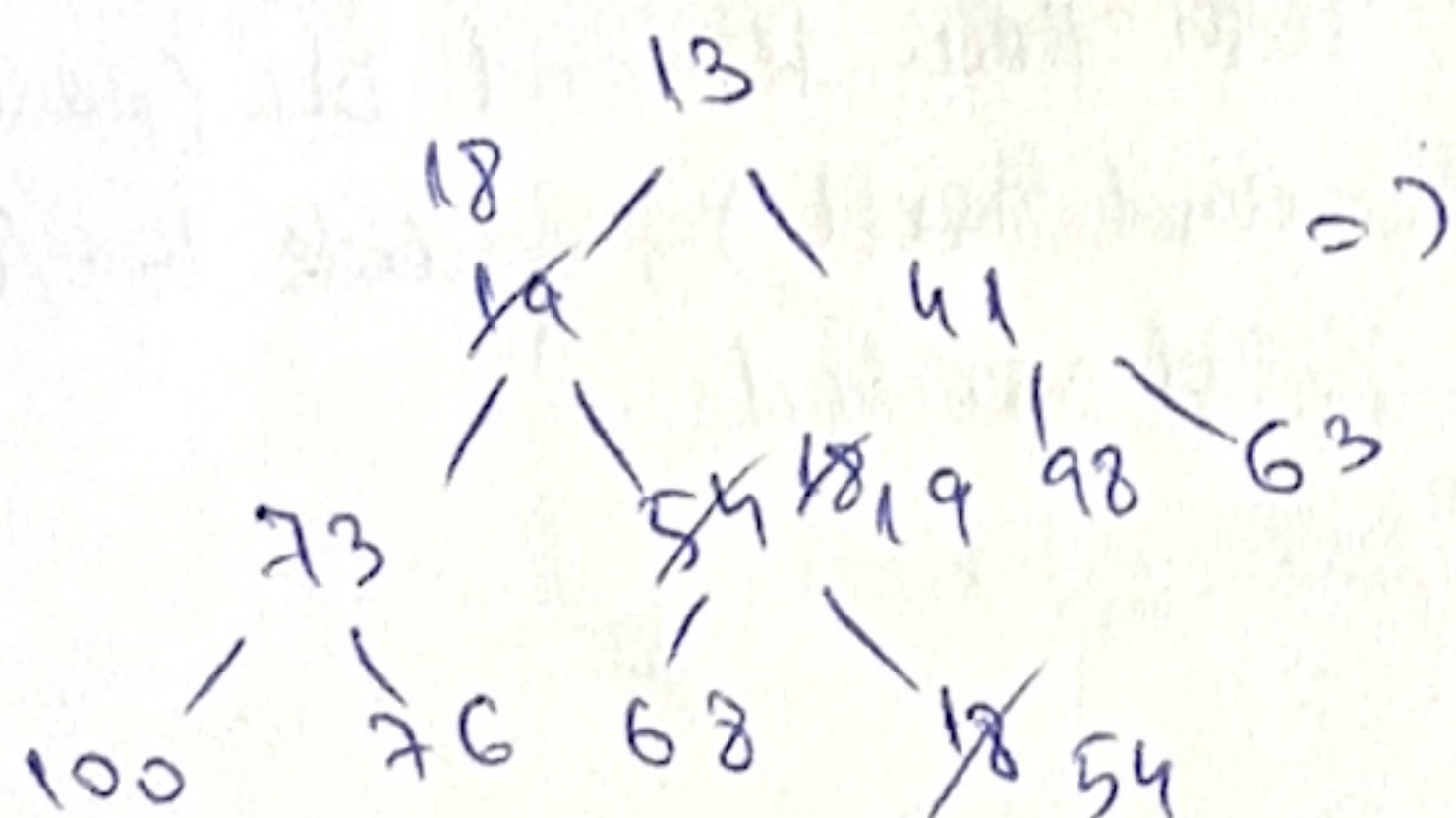


⇒

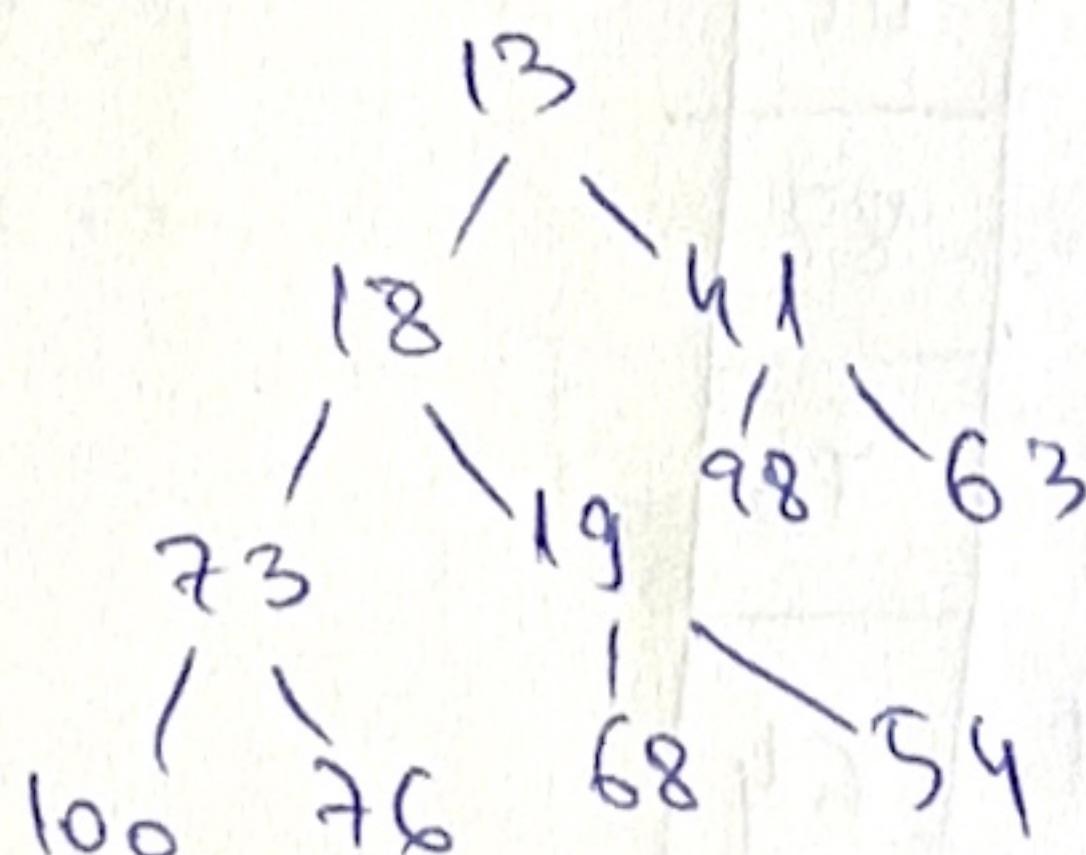


min heap (valid).

• add 18

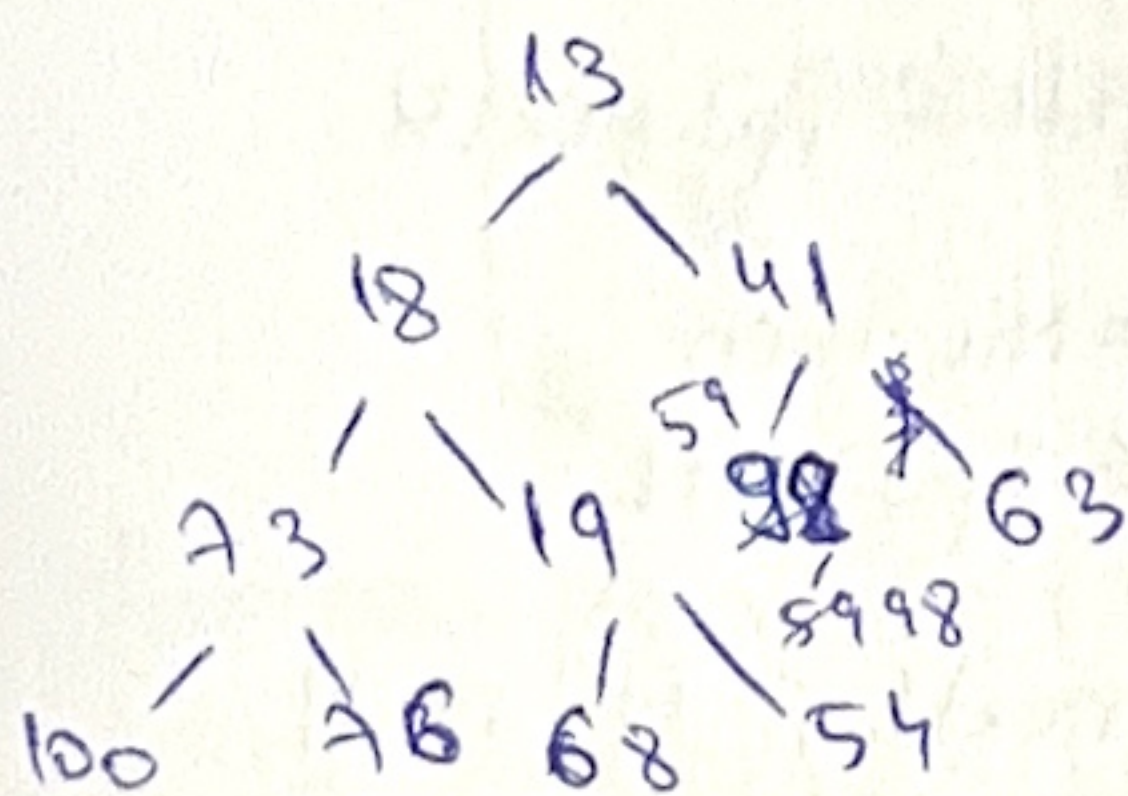


⇒

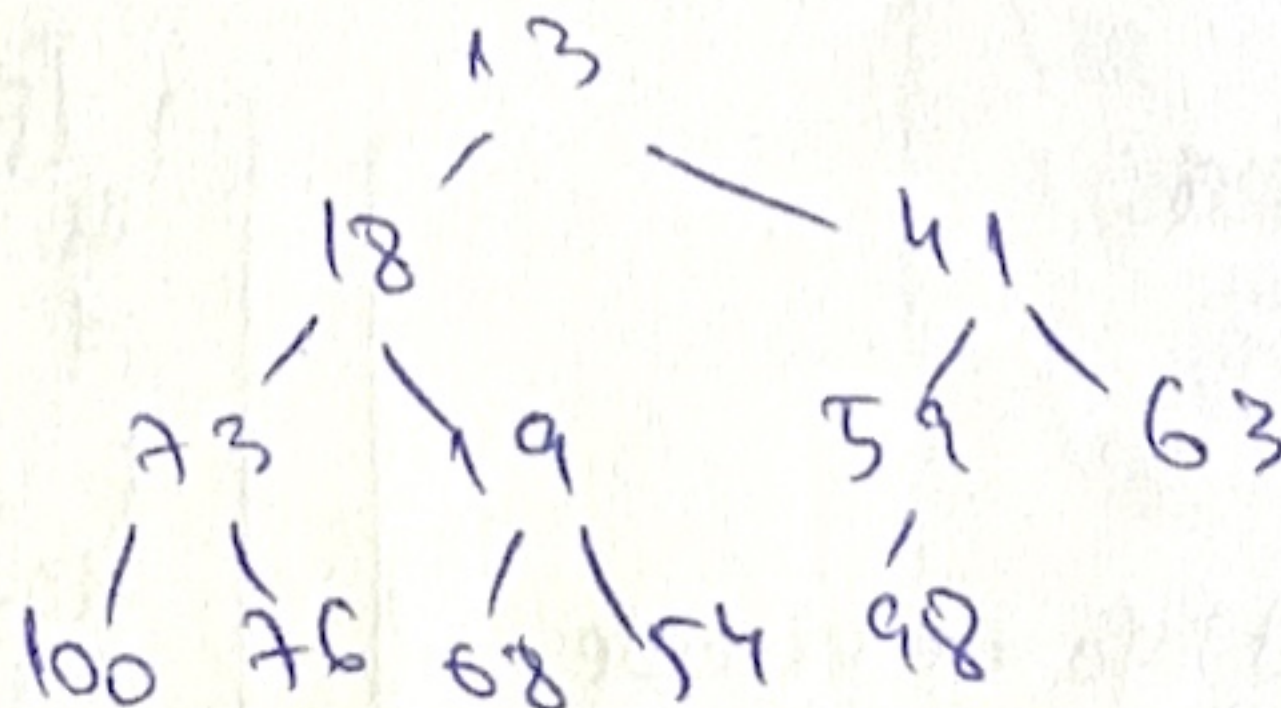


min heap.

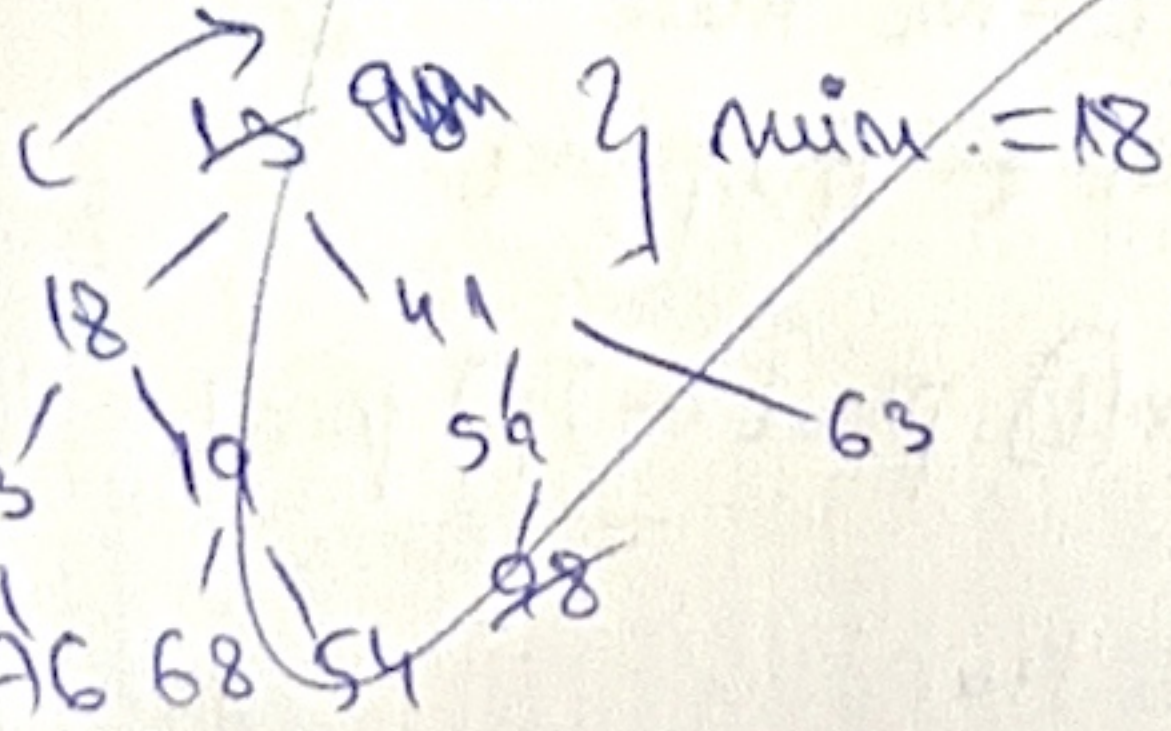
• add 59



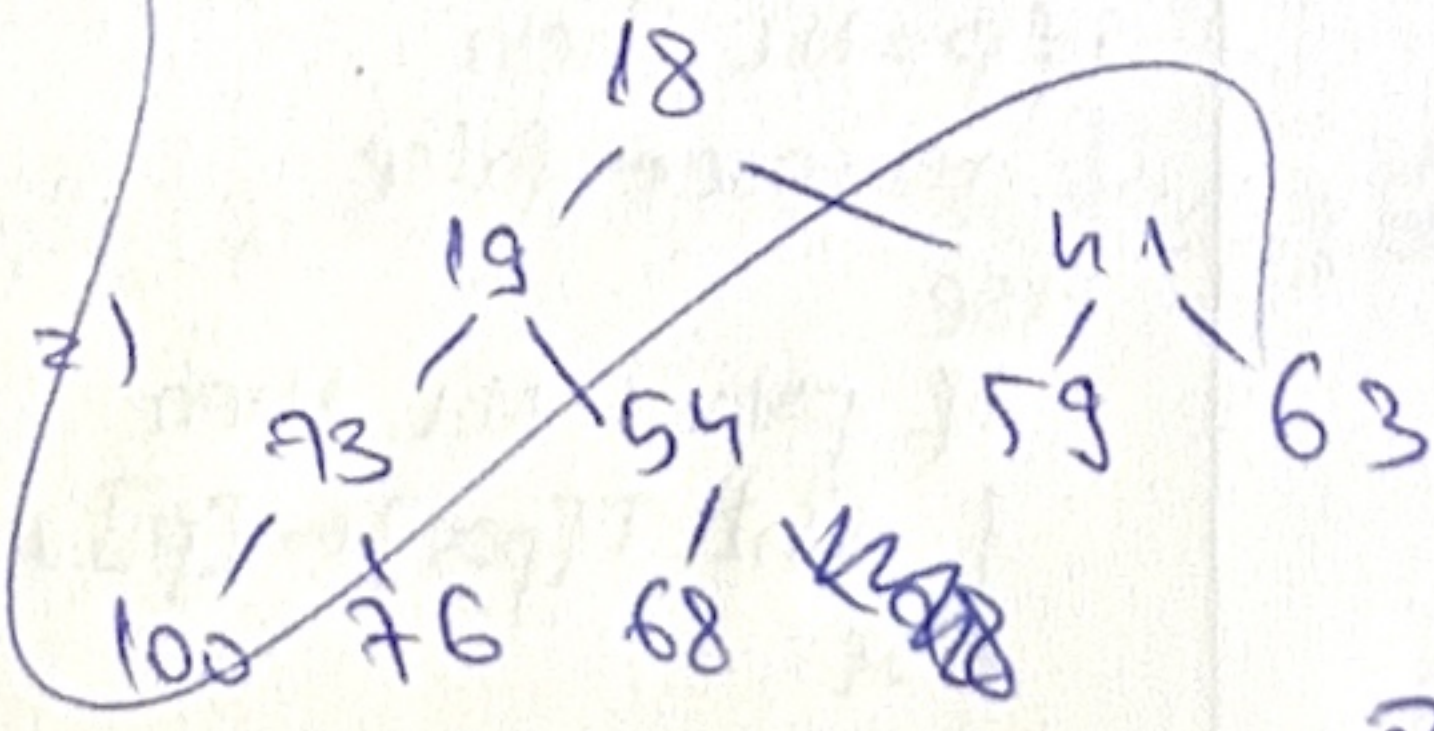
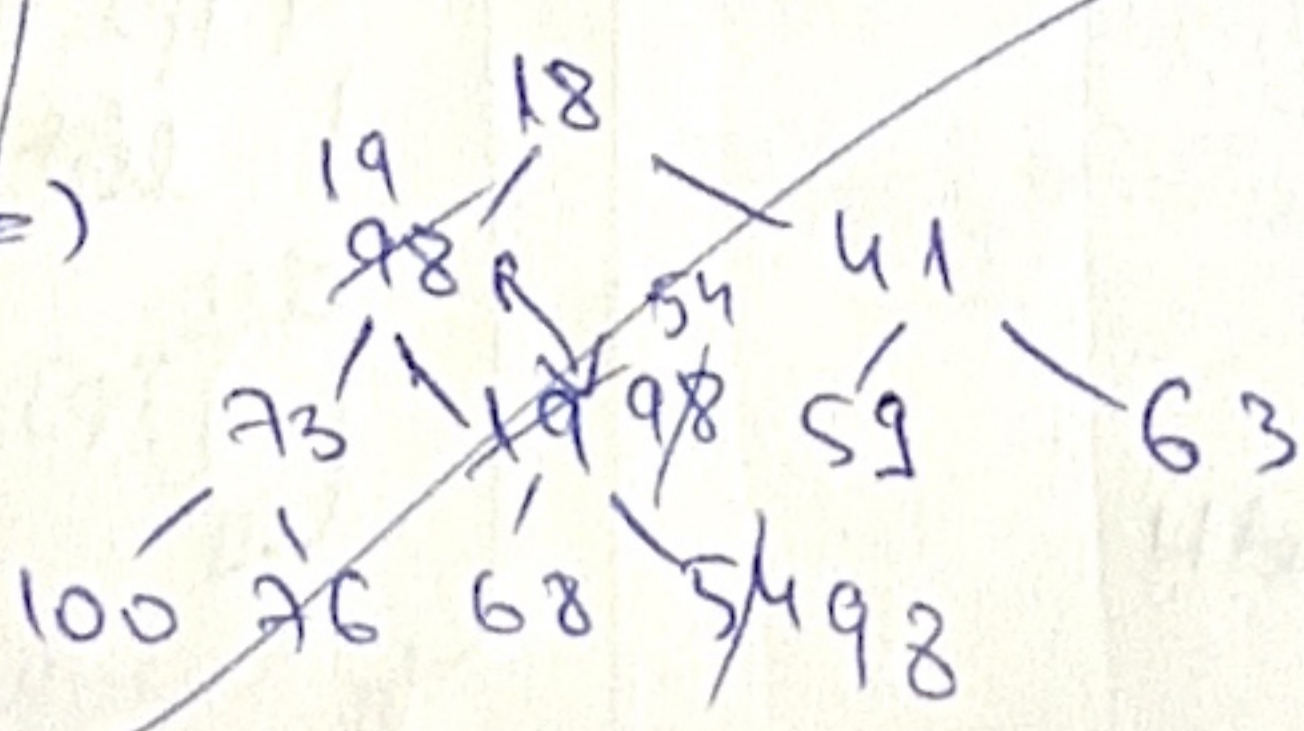
2)



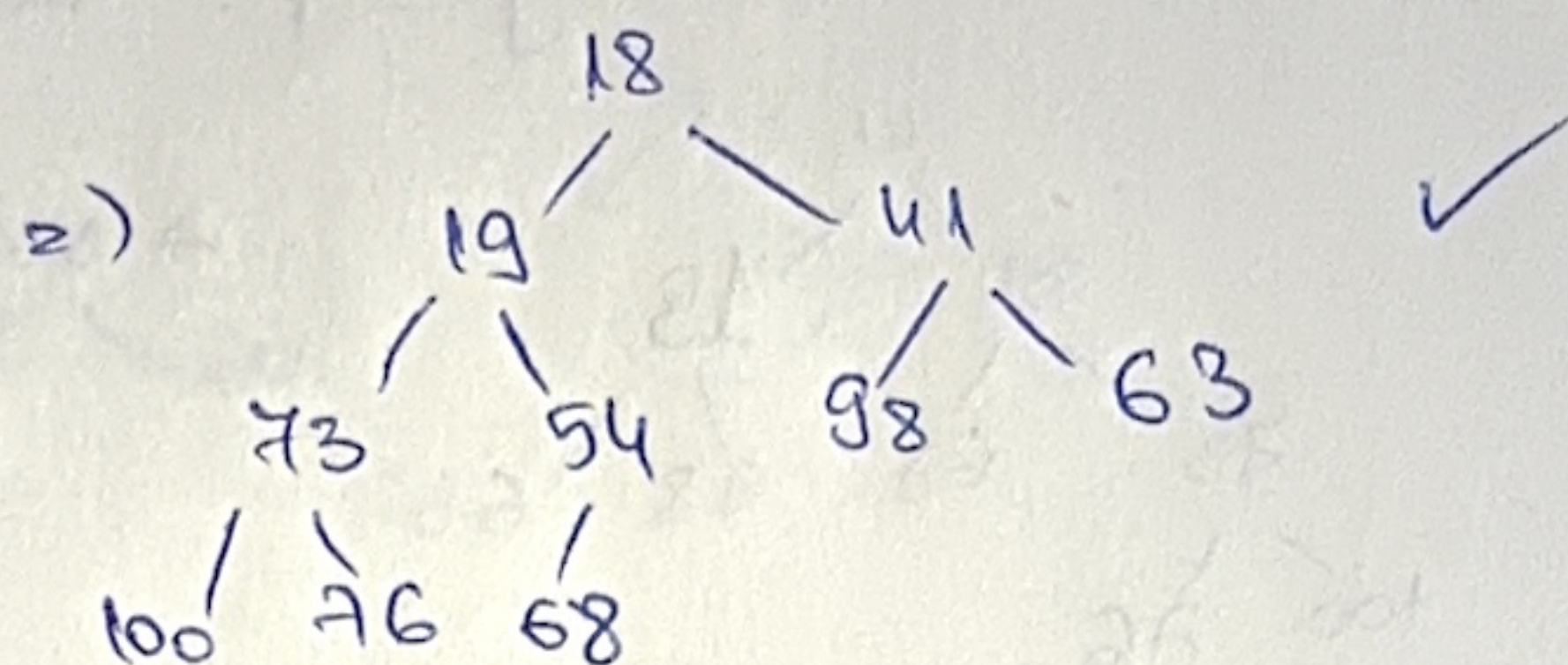
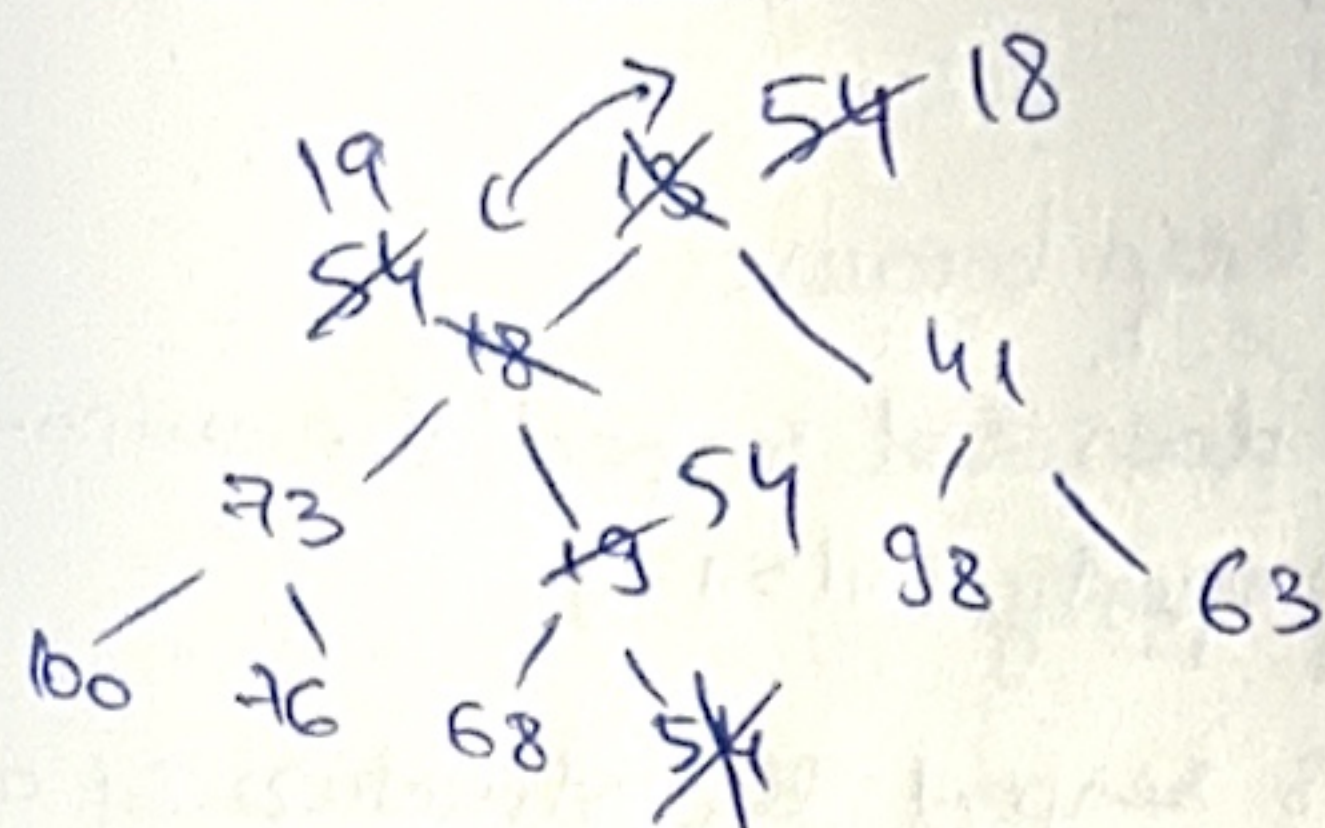
• remove elem (root, 13):



⇒



• remove 13 (root)



- ③. DS $\rightarrow$ search, insert, remove $\rightarrow$ same complexity as a HT.
 $\rightarrow$ Iteration order = insertion order (not sorted or anything)
 $\rightarrow$ iterator operations: $\Theta(1)$

$\Rightarrow$ LINKED HT.

Representation:

Node:

info: $\uparrow$ Key

nextH: $\uparrow$ Node

nextL: $\uparrow$ Node

prevL: $\uparrow$ Node

Linked HT:

m: Integer

T: ($\uparrow$ Node)[]

h: TFunction

head: $\uparrow$ Node

tail: $\uparrow$ Node

* Remove needs prev node in insertion-order list $\Rightarrow$ DLL (SL would be inefficient),

Complexity: BC = $\Theta(1)$
 WC = $\Theta(m)$

$\downarrow$
 Search for $\odot$ in the HT using (nextH), remove node both from HT and DLL (prevL and nextL), update tail/head if needed.

ex: m=10

$h(12) = 12 \% 10 = 2$

$h(41) = 1$

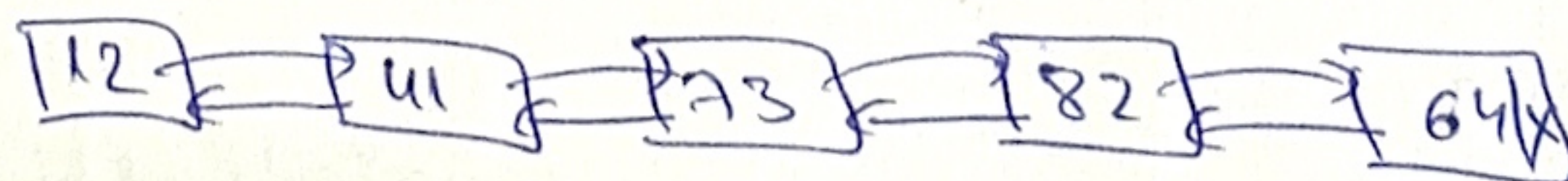
$h(73) = 3$

$h(82) = 2$

$h(64) = 4$

0	12
1	41
2	82
3	73
4	64

Insertion-order DLL:



Remove:

function remove(lht, e) as

pos $\leftarrow$ lht.h(e)

p $\leftarrow$ lht.T[pos]

prevH $\leftarrow$ nil

while p $\neq$ nil and [p].info $\neq$ e execute

prevH $\leftarrow$ p

p $\leftarrow$ [p].nextH

if p = nil then

remove $\leftarrow$ false

else

if prevH = nil then

lht.T[pos] $\leftarrow$ [p].nextH

else

[prevH].nextH $\leftarrow$ [p].nextH

if [p].prevL = nil then

lht.head $\leftarrow$ [p].nextL

else

[p].prevL.nextL $\leftarrow$ [p].nextL

if [p].nextL = nil then

lht.tail $\leftarrow$ [p].prevL

else

[p].nextL.prevL $\leftarrow$ [p].prevL

remove $\leftarrow$ true

1) Queue using 2 stacks. Take stack ADT for granted.

→ Stack operations:

initially)

- init(s) // creates an empty stack
- push(s, e) // adds e on top of the stack
- pop(s, x) // remove + return the top element; throws exc. if empty
- top(s) // returns top element; throws exc. if empty
- isEmpty(s) // returns true if stack is empty, false otherwise

→ Queue representation:

Queue:

SIn: Stack // newly pushed elems.

SOut: Stack // elems ready to be popped in FIFO order.

→ Queue operations:

init(q)

push(q, e)

pop(q)

front(q)

isEmpty(q)

"transfer".

→ Queue implementations:

subalgorithm init(q) is

- init(q.sIn)
- init(q.sOut)

subalgorithm push(q, e) is

- push(q.sIn, e)

subalgorithm reverseOrder(q) is

- while not isEmpty(q.sIn) execute
 - x ← pop(q.sIn)
 - push(q.sOut, x)

function pop(q) is

- if isEmpty(q.sOut) then
 - reverseOrder(q)
- if isEmpty(q.sOut) then
 - @throw exception
- pop ← pop(q.sOut)

function front(q) is

- if isEmpty(q.sOut) then
 - reverseOrder(q)
- if isEmpty(q.sOut) then
 - @throw ...
- front ← top(q.sOut)

function isEmpty(q) is

- isEmpty ← isEmpty(q.sIn) and isEmpty(q.sOut)

→ Complexity (Bc, wc, total amortized); $\Theta(1)$ stack op.

• init: Bc = wc = $\Theta(1)$ = total amortized.

• push: Bc = wc = $\Theta(1)$ = total am.

• pop: Bc = $\Theta(1)$ → sOut is not empty ⇒ pop directly

wc = $\Theta(m)$ → sOut is empty ⇒ transfer ⇒ pop
(move m elem.)

⇒ total am: $\Theta(1)$

• front: Bc = $\Theta(1)$ → n pop.

wc = $\Theta(m)$

⇒ total am: $\Theta(1)$

• isEmpty: Bc = wc = total am. = $\Theta(1)$

Total amortized = long seq. of operations ⇒ divide total cost by ~~no. of~~ nr. of operations

Our case: ~~push~~ each elem. → push to sIn once → pop from sIn once
→ move to sOut once
→ pop from sOut once $1+1+1+1=4$

m elems ⇒ 4m ⇒ $\Theta(m)$ = total cost.

nr. operations: $\Theta(m)$

⇒ Total amortized = ~~no. elems~~ $\frac{(\text{total cost})}{\text{nr. op.}} = \frac{\Theta(m)}{\Theta(m)} = \Theta(1)$

Particular case:

6 operations {
push(1)
push(2)
push(3)
pop() → transfer: 1 → sOut
pop() → no transfer: ① 2 → sOut
pop() → no transfer: ① 3 → sOut
pop() → no transfer: ①

⇒ total cost = ③

⇒ 3 pops from sIn
+
3 pushes to sOut
+
1 pop (final)

⇒ total cost: ⑦

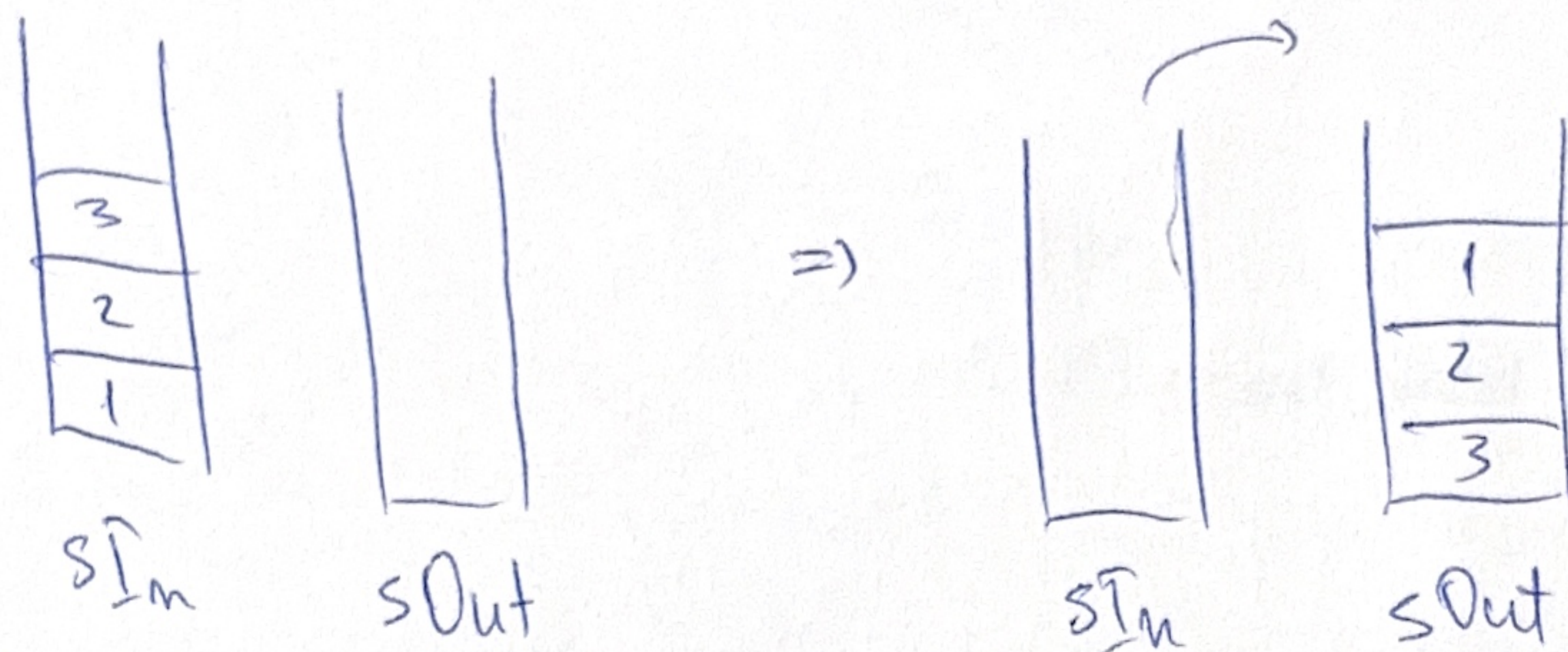
Remaining pops:

Total cost (final): ③ + ④ = ⑦ = 12

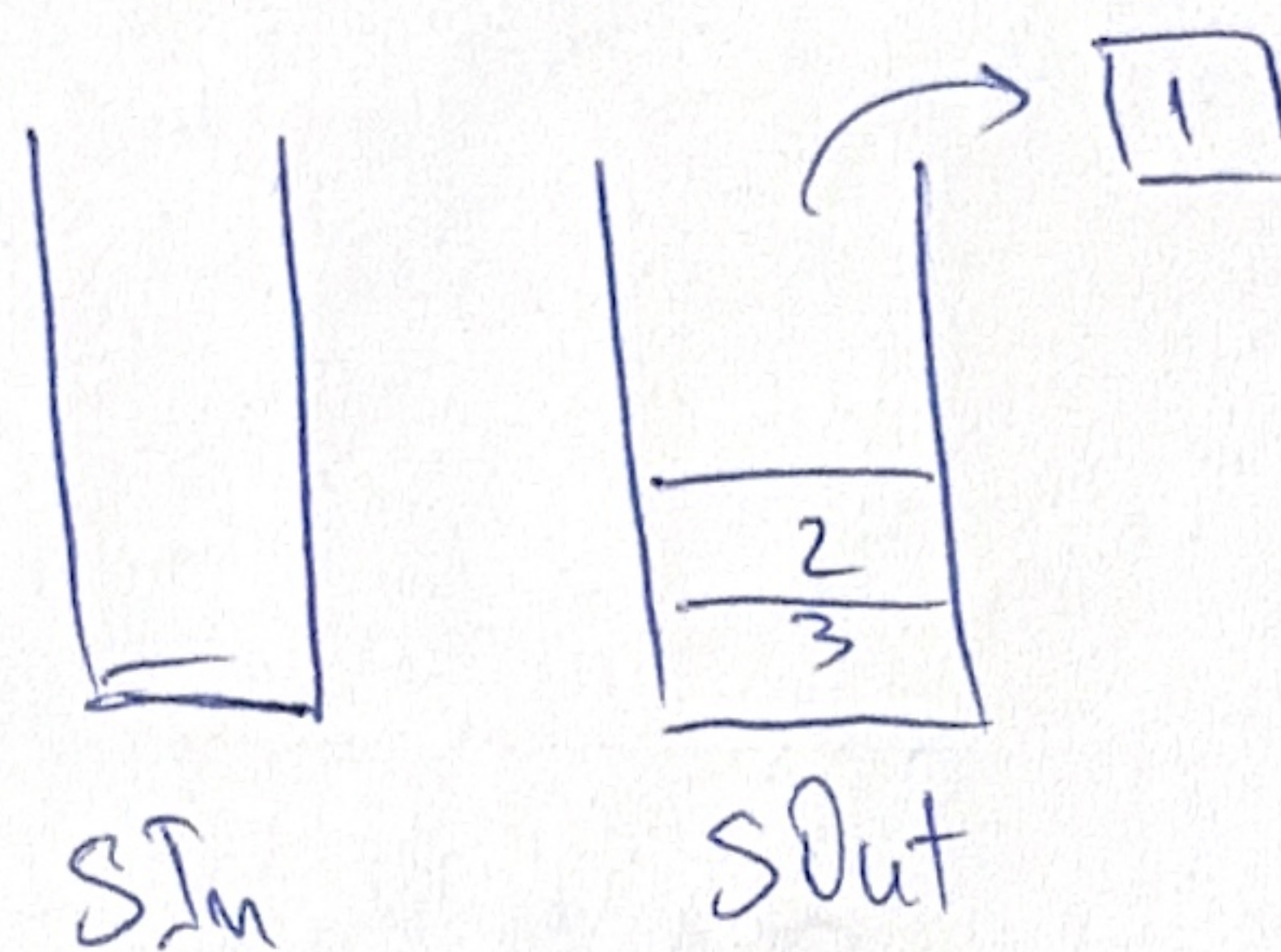
Nr. of operations: 6

⇒ total amortized = $\frac{12}{6} = 2 = \Theta(1)$

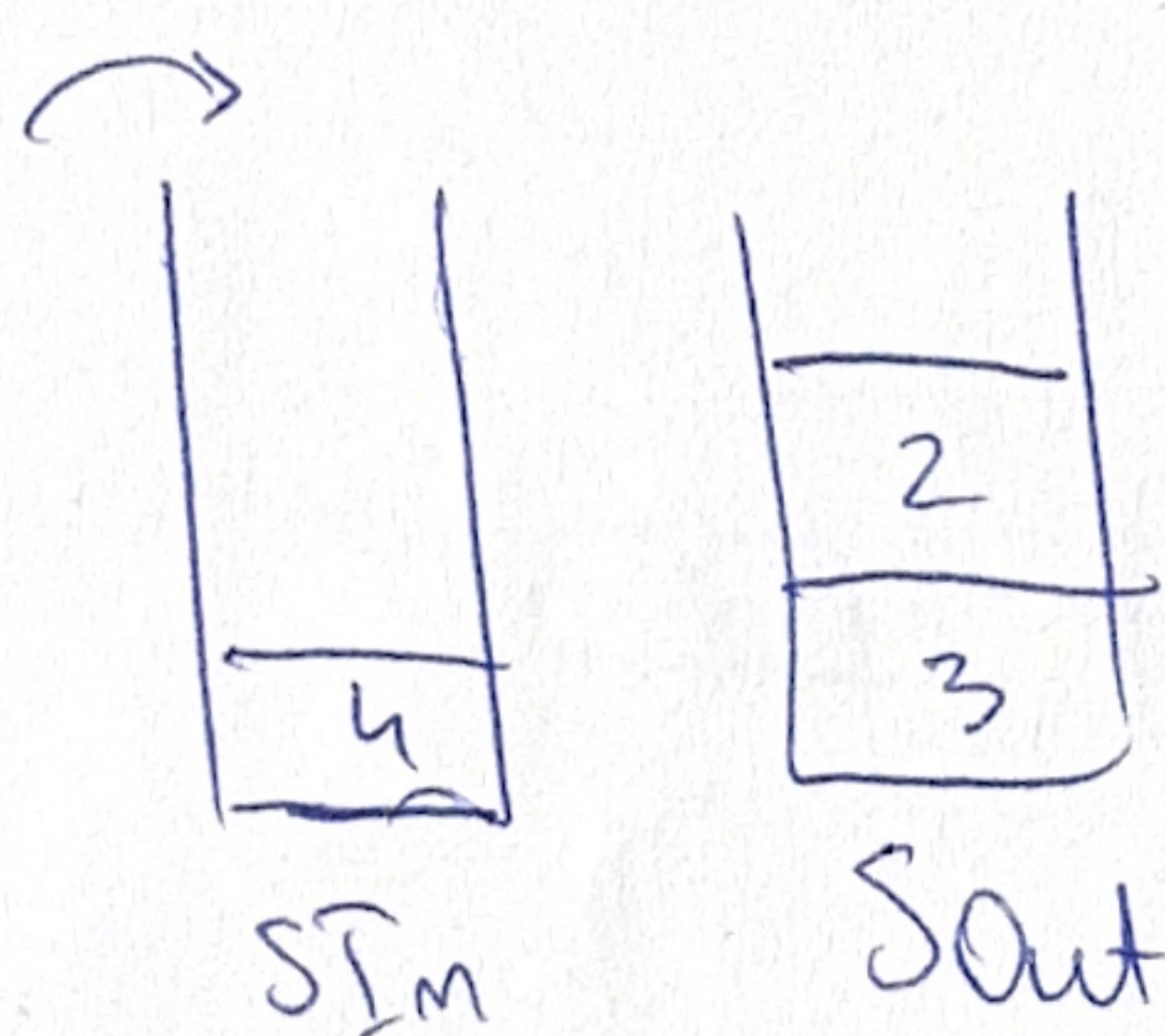
→ 1,2,3 in queue



after 1 pop
from QUEUE)



after push
elem 4)



after 1 pop →

